What's new

Palomar – a registry of Lean verified mathematics

Terence Tao · 18 August 2026

https://terrytao.wordpress.com/2026/08/18/palomar-a-registry-of-lean-verified-mathematics/

tao2typ · converted to Typst 2026-10-14 (UTC)

In recent months there has been a proliferation of AI-generated proofs of various old and new results, some of which have been formalized in the proof assistant language Lean. However, checking that a given Lean repository actually proves the claimed statement is somewhat non-trivial, especially for an audience which is not expert in the use of Lean: one has to first check that the claimed formal Lean statements have proofs that typecheck, that the proofs do not contain any "cheats" such as adding additional axioms, and that the formal statements also match (in a semantic sense) the informal description of the claimed results.

To help bring some clarity to this situation, I am happy to announce that Palomar registry of Lean verified mathematics, which is an initiative incubated by the Lean FRO and by ICARM, is now open for submissions. I am serving in several roles on this registry, including on the scientific advisory board, together with Jeremy Avigad, Matthew Ballard, Jaume de Dios, Nestor Guillen, Bryna Kra, Kim Morrison, Ravi Vakil, and Akshay Venkatesh.

A detailed motivation for Palomar can be found here, and further information about Palomar can be found here. A zeroth approximation of what Palomar intends to be is the analogue of a preprint server for Lean proofs. More precisely, Palomar (which is named after the astronomical observatory) is a registry of external Github repositories (or more precisely, "snapshots" of such repositories, as represented by a specific Github commit) containing Lean code adhering to the current best practices for such formalizations, in particular containing

- A "challenge file" containing a short, human readable description in Lean of the results claimed.
- A "solution module" containing an (arbitrarily long) proof of the results claimed in the challenge file.
- A "formalization.yaml" file describing the results in informal language, and also containing a number of other relevant metadata and disclosures.

(There are also some additional technical requirements for the repository which I will omit here.) If a snapshot of a repository is submitted to Palomar, it will check both (a) that the solution module typechecks and proves exactly the results claimed in the challenge file, and that (b) the informal description of the result in the formalization.yaml file appears to match the result claimed in the challenge file, and that the repository meets various minimal standards required for a registry entry. The first check (a) is purely mechanical, using the Lean tool Comparator; the second check (b) is non-deterministic, being performed by a large language model. If a repository passes both checks, it can be registered on Palomar. It is worth stressing that the checks in (a) and (b) fall well short of what a proper human peer review of a submission for novelty, interest, and accuracy would give; in particular, Palomar is **not** a peer-reviewed journal.

The submission process is thorough, but achievable: as a test, I successfully managed to submit my own recent formalization of the proof of Sendov's conjecture to Palomar, and also plan to submit some older formalizations to the registry soon.

In any event, the registry is now open for formalizations of both old and new results. Submissions (whether human-generated, AI-generated, or some mixture of both) are welcome; please read the (somewhat detailed) instructions here before starting a submission. (I will however note that modern AI agents are quite helpful in assisting with the mechanical details of the submission, though a human review is still strongly recommended.)

Discussion and feedback on Palomar will occur on this Zulip channel.

Comments (47)

Anonymous · 18 August 2026

cool

Anonymous · 18 August 2026

> the second check (b) is non-deterministic, being performed by a large language model

Shouldn't this be just preliminary? I think submissions should have an additional, human-performed level of verification.

> **Terence Tao** · 19 August 2026
>
> As stated above, Palomar does not perform human review of repositories, which will not scale to the volume of repositories we envisage processing. (This is similar to the arXiv, which performs minimal checks of acceptability for the preprints it accepts, but also does not perform human review.) However, we would be fine with a third-party service performing additional review on top of the minimal checks that Palomar performs.
>
> **Tanay Wakhare** · 7 September 2026
>
> The main issue here is the relatively small number of people with experience reading/writing good quality Lean. There's less than 1000 core contributors to mathlib, and it's not common in industry. LLMs can catch some of the misformalization errors, with the caveat that they often still miss errors.

Anonymous · 19 August 2026

As GitHub is becoming less reliable every year, have you considered alternate git forges or hosting the repositories yourself instead of relying on external companies to keep their services available in the future?

> **Terence Tao** · 19 August 2026
>
> We do not have the resources to host and maintain repositories directly, but would be open to expanding the whitelist of approved repository hosting services beyond Github if there is sufficient demand for doing so.

David Bevan · 19 August 2026

A very quick look at Palomar suggests that it would be much more helpful if each submission required (a) a meaningful Title, and (b) a (well-written) Abstract explaining what is proved — just as one sees in arXiv. At present, the titles and descriptions do not enable me to understand what many of the submissions achieve.

To be honest, I'm rather shocked that this went live without a requirement of this sort. (Who is the main interface actually supposed to be for if it's impossible to work out what entries are about?)

> **Terence Tao** · 19 August 2026
>
> I'll pass this suggestion on to the maintainers. Currently we rely on the `project.name` and `project.description` fields of the repository `formalization.yaml` to provide the informal description of the project, although this does not perfectly map on to the title and abstract of a preprint because the claims being registered may only form a subset of the content of the

repository, rather than represent the full repository project itself. So there may be a need to add separate title and abstract fields, but this will need some discussion to implement properly.

> **Anonymous** · 20 August 2026
>
> This is not related to your project, but I have been considering implementing a standard that would formalize a bit this kind of process, pinning a claim to a computation. See claimpins.org.

Anonymous · 19 August 2026

Such a good idea! Do you think that there is a chance of integration with Andromа? If so, please get in touch with viktor@andromа.org

ygtisik · 19 August 2026

nice idea overall for setting a level. I work on whataifound.org to maintain a documented registry of AI usage and contributions in math and science. Palomar will definitely be something we integrate for data checks and validation.

Anonymous · 19 August 2026

Is there consideration of requiring or at least marking submissions that include a human readable explanation of the proof(which can be higher level than a normal paper since the proof is formalized alongside it). I'm concerned that the machinery behind them will become much less accessible for reuse.

> **Terence Tao** · 19 August 2026
>
> This registry is not designed to assess informal proofs (see item 3 of https://palomar-registry.org/about#what-palomar-is-not). However, we have no objection to having third-party services that build upon the registry to perform assessments of this type.

Lars Warren Ericson · 19 August 2026

How would you handle this repo with this arXiv writeup and this Lean proof code directory. It formalizes an entire paper which has multiple definitions and theorems. In general how would you approach Lean formalizations of whole papers, especially very long ones with hundred of results?

> **Terence Tao** · 19 August 2026
>
> We have a hard cap of 1000 lines (and 100KB) on the size of the challenge Lean file, which we very much want to keep human-readable (and AI-reviewable). (We in fact have a preference for challenge files that are significantly shorter than this hard cap, e.g., 300 lines.) For a large repository with multiple theorems which cannot all be stated in Lean within this cap, this would mean that multiple registry entries would be required to register separate statements for each major results.
> If the results require extensive definitions to import that are not currently in Mathlib and whose constructions are too large to fit within the cap, then another option is to set up the required definitions in a Tau Ceti repository, which can then be imported as a dependency for the challenge file.
>
> > **Lars Warren Ericson** · 23 August 2026
> >
> > Regarding Tau Ceti, the workflow there seems to be: (a) propose a topic, (b) get it approved and allocated to you, (c) input an AI workload to formalize, (d) AI formalizes the work hands-free, (e) it gets saved publicly if that works. That is quite different from storing the work

already done for a difficult formalization. Lean Pool, as an option, saves submitted Lean code. It does not save documentation or articles written about the code. It only indirectly points back to the original repo, through GitHub cloning links. It actively golfs the code, which can eliminate comments and original proof structure in favor of more opaque tactics like “simp all”, which can erase or restructure the originally written proof from the source article. So: neither TauCeti nor Lean Pool is an option for building on authored repos with Lean formalizations. The only option for Palomar is to vendor all dependent repos into the submission. Please correct me if I’m wrong about this.

> **Vasily Ilin** · 10 September 2026
>
> I am the maintainer of Lean Pool. Do you have any suggestions for how to make it better or more useful?
> -Vasily Ilin

Anonymous · 19 August 2026

Do you think it is acceptable to register important results on Palomar before a preprint is submitted (while the writing is still in progress)?

> 
> **Terence Tao** · 19 August 2026
>
> I have (reluctantly) come to the conclusion that authorship of the different stages of a proof development cycle – generation, verification, exposition, publication, and canonicalization – will become increasingly decoupled, that an author may be responsible for one part of this cycle but not be involved in another. In particular, much as we already normalize the submission of a preprint to a server such as the arXiv before formal publication, or the publication of a paper before the definitive textbook containing the result is written, we will increasingly have to also permit the registration of formal proofs before a preprint written to professional standards is available. I do not view this decoupling of authorship as ideal – there are advantages to having a single group of authors be responsible for a proof throughout its development, much as there are advantages to having a single group of parents raise a child from conception to adulthood – but non-traditional authorship arrangements may be manageable as long as the incentives are such that the separate groups of authors can each be recognized for their particular contributions to the proof development cycle. (In particular, we may need a more explicit process to “hand off” the responsibility of authorship from one group to another in such cases.)

Anonymous · 19 August 2026

How likely is it that AI can start reading through textbooks and autoformalizing results by following the arguments line by line (possibly up to issues with canonicalizing with the current Lean library)? For example Stacks Project.

> 
> **Terence Tao** · 19 August 2026
>
> There has been a very significant improvement in autoformalization capabilities in the last six months alone, at least if one focuses on the narrow goal of *only* certifying that a formal proof exists. I expect that (at least in some subfields of mathematics) a large fraction of textbooks and papers will soon be technically autoformalizable in that they can produce formalized versions of the statements of the main results of such texts, and provide typechecked proofs, though the proofs may not literally follow the informal proof “line by line” and may be inefficient, not human-readable, or otherwise not completely satisfactory in terms of code quality.

Anonymous · 19 August 2026

Terry, this is a very nice project!

I run a site called TheoremDB, which approaches some of the same questions from a different angle and may be of interest to you. I think there is a large engineering design space here that is worth exploring, so seeing how another site approaches the problem may yield some useful ideas for Palomar.

Some specific differences:

TheoremDB gives a theorem an identifier independent of any particular repository or Lean declaration, so several proofs, formalizations, computations, and failed attempts can be collected under the same statement. This separates "has this artifact been checked?" from "what is already known about this theorem?"

I am especially interested in what this permits for "negative traces." Human mathematical literature preserves only a small fraction of this material. With machine mathematics, such traces can be produced as a routine by-product of search, stored systematically, and returned to the next agent before it repeats the same work. This gives us reusable research memory at a scale that was previously impractical. Here is an example packet; the underlying artifacts are in the Work tab.

TheoremDB exposes its agent workflow through MCP. Palomar organizes submission around a GitHub repository, so this gives agents a somewhat different way into the system. An agent can fetch the exact statement and previous work, check a private Lean draft, submit the exact source that passed, and later check its verification status. A Palomar MCP could plausibly expose its existing preflight and submission process in the same way. I also created a custom GPT as a ready-made client, making it easier to create entries with very little setup.

One further difference is repeated private checking before submission. Palomar already uses GitHub Actions for public verification. TheoremDB also keeps Lean environments ready for private draft checks, which makes iteration considerably faster. If Palomar eventually wanted this kind of draft lane, it could sit before the current public verification process.

Anyway, I think Palomar is a wonderful and rigorous initiative. There is a large engineering design space here, and I think it will be valuable to see several approaches explored.

Anonymous · 19 August 2026

"The submission process is thorough, but achievable:"

This is AI.

> 
> **Terence Tao** · 19 August 2026
>
> Sorry to disappoint any AI detectives, but the text here was human-generated. (The use of a "but" after a comma (or the rhetorical device of antithesis in general) predates the advent of AI by several millennia, although AI models likely have trained extensively on that particular rhetorical pattern in the literature, most infamously in the "Not X, but Y" construction. But this does not justify affirming the consequent.)
>
> As a side note, integrating AI workflows into WordPress directly currently requires more authorization of my AI agents over my computer than I am currently willing to grant, and I don't see a major efficiency gain in delegating the writing of short blog posts like this to AI to be worth the various costs. I have managed however to use AI to improve Luca Trevisan's old LaTeX to WordPress HTML converter Python script, which I now use for lengthier blog posts (for instance, my version of the script now handles images, which I previously had to input by hand); and it has also created a tool to semi-automatically repair mangled text in comments arising from the quirks in WordPress's hybrid LaTeX/HTML parser.
>
> *(All uses of rhetorical antithesis in this comment were also human-generated.)*

Vinicius Rodrigues · 19 August 2026

Interesting project!
Last week we sucessfully used GPT-5.6 Sol to solve Wallace's question, a 73 year old problem on the structure of topological groups and semigroups. We also produced a formal proof. This morning I submitted it to Palomar to try it out, and it was accepted in less than a hour. It will be interesting to see how the project evolves!
Cheers!

Anonymous · 20 August 2026

Is a one theorem to many formalizations, or perhaps simpler to manage one formalized theorem to many formalized proofs supported?
At some point, with a large enough database, it would be interesting to use the formalized proofs to study measures of quality. Some fuzzy and hard to define, like elegance, some more objective, like containing lemmas that are used in many other proofs. Perhaps the comparative study could detect lemmas, or proof patterns, which applicability has been overlooked.

> **Terence Tao** · 20 August 2026
>
> Palomar is organized around individual theorems that have a designated proof, although the proof itself is only checked to establish a type-identical statement to the claimed theorem, and no further analysis of the proof is provided. So a theorem with N different formal proofs would give more or less identical Palomar registry items. But I certainly agree that other qualities of formal proofs should be studied systematically; this is out of scope of this particular registry though.

Lars Warren Ericson · 20 August 2026

If a submission requires revision, e.g. for a metadata update, is the right workflow to push "Abandon this submission" for the prior submit after revising? The workflow available for revising a submission is for registered and accepted submissions, not new ones.

> **Terence Tao** · 20 August 2026
>
> Starting a new submission for the same repository automatically closes any previous open submissions, so it is not necessary to explicitly abandon the previous submission. (We don't yet have a separate mechanism for updates that only change metadata, so unfortunately any such change will need to rerun comparator etc.)
>
> > **Lars Warren Ericson** · 20 August 2026
> >
> > Today I registered bridge theorems between 3 presentations of Scott domain theory (lattices, information systems and neighborhoods). It took 3 tries. First it asked me to revise the bridge theorems. Then it said everything was OK but my metadata. I initially had one repo for 1972, 1980/81 and 1982 papers and then I split it into 3 because of complexity and then I wrote a 4th with the bridge theorems. The first 3 repos were formalisations and the bridge theorems I have not seen elsewhere. Formalization and novel get different keywords. I had to describe this provenance exactly right in the metadata. Then I submitted again and the reviewer engine failed to complete 2 times before succeeding. Finally I got the Register button. Which is awesome! But history of why each submit got changed is buried in the final inducted repo, which is OK but not as easy to track in your system if you want to look back and see for example what are the most common causes are of resubmit. Also, versus an earlier comment, getting the bridge theorems accepted is in a way a hack that gets the underlying formalizations into the system. All told around 90,000 lines of Lean code. But the bridge theorems under 400!

Anonymous · 21 August 2026

This project makes me realize that at some point in the coming crisis, all our papers and books will probably get autoformalized. It would be not great if it happens in a complete chaos. I am not super happy with the idea of having anyone (or several people) autoformalizing my papers and their machines possibly finding mistakes. Ideally I would prefer having a colleague trying to understand it and possibly pointing mistakes (but let's be realistic: my existing papers get seriously read by maybe 2-3 people so far, and it will not get better with the flood, though I have had the chance to have some serious referees on a few past occasions). So it's not inconceivable to put them here (if co-authors agree), knowing that it's at least managed by the community. Let me specify that I am not at all an AI-enthusiast, much to the contrary, neither a supporter of autoformalization, and I am not willing to contribute myself in the coming flood by using LLMs.

> **Anonymous** · 26 August 2026
>
> A rigorous verifiable Theorem in a strict Kernel built by LLM is still better than the text generated by the LLM for the journal. I am neutral and I believe what is important is guardrails.

Lars Warren Ericson · 21 August 2026

The Palomar registration function is broken. I have a validated submission that is stuck waiting for operator attention since 2026-08-20T18:44:33Z. I put a note on the Zulip Palomar channel: #Palomar > Registration queue jammed @ 💬

Lars Warren Ericson · 23 August 2026

If a new theorem to be registered depends on a previously registered theorem, and it is in a different github repo from the first one, what is the correct pattern to build on a previously registered result? Should it involve placing both in TauCeti? Or can one claim build on the Palomar copy of the first repo? It would be very helpful to provide a formal guideline for the approved and respected methodology.

> 
> **Terence Tao** · 23 August 2026
>
> Palomar's whitelist for library imports for the challenge file currently consists of Mathlib, CSlib, and Tau Ceti; in particular it cannot use arbitrary definitions imported from arbitrary repositories, which is a very intentional choice on our part. However, the solution file may use external imports beyond this whitelist.

alonsoquixano · 25 August 2026

Thank you for the pointer. A few questions/comments:
– One clicks on an article, clons the corresponding GitHub repository, and then tries to verify the proof using one's own Lean installation and the instructions https://lean-lang.org/doc/reference/latest/ValidatingProofs/#validating-proofs?
– There was some talk of using converters. I assume this refers to translating informal theorems into Lean. What about the reverse direction — for example, converting a theorem in Lean into an informal version?
– A general suggestion for the blog, from someone who is only comfortable mainly with ZFC and first-order logic: how about a post explaining the advantages of Lean, perhaps with a comparison with Isabelle (or Metamath)?
Uwe Brauer

Anonymous · 26 August 2026

My Collatz Palomar submission is now registered.
Title: polylogarithmic descent for almost all collatz orbits in natural density
https://palomar-registry.org/entry?id=PALOMAR-2026-08-26-000005&version=1
The community was incredibly helpful, especially Kim Morrison, who was remarkably available and supportive throughout.
As an independent researcher and a software developer for over 2 decades, I feel genuinely empowered by this. I've faced journal desk rejections, so I'll allow myself one small and slightly petty victory lap here.
Thank you Dr Tao and the whole Lean FRO and ICARM for bringing Math to coders now.

> **Anonymous** · 29 August 2026
>
> Can you point us to the paper?

Anonymous · 28 August 2026

Has anyone seen the purported AI solution to Hopf's problem that Levent Alpöge posted to twitter a few days ago? There are now two separate Lean formalizations but IDK what reservations might exist about them. They were apparently also done by AI, though the problem statement itself was done by humans some time ago.
More info at the comment thread: https://mathoverflow.net/a/514604/129086

Lars Warren Ericson · 29 August 2026

I have been using Palomar to validate and register a set of formalizations of Dana Scott's presentations of a topological model of untyped lambda calculus, and bridge theorems between the presentations. I was impressed by the ability of the Palomar review to detect differences between the informal English statement of a claim and the Lean 4 formal statement. Palomar uses OpenAI's frontier general purpose LLM GPT-5.6, and a thorough set of policy prompts. I am proud of my successful registrations. Do you agree with Kim Morrison's belief that "Palomar does not "review" code, it only rejects or doesn't reject it using some simple fubrics. :woman_shrugging: Please don't pretend it is some editorial stamp of approval: your work can be in Palomar and still might be crap and irrelevant!"? If so, what percentage of the 116 results registered to date would be found to be "crap and irrelevant" upon your review or the review of a mathematician of similar skill and training?

> **Terence Tao** · 30 August 2026
>
> As stated in Palomar's about page (see in particular "What Palomar is not"), Palomar does not perform actual peer review on its submissions; the automated checks it supplies can flag some obvious alignment issues with a formalization, but is not a substitute for genuine human review, which we do not have the resources to conduct on the volume of submissions received.
> There are plans by other parties to implement a "Palomar overlay journal", which would conduct peer review on Palomar repositories submitted to that journal, but this would be a separate project. We are intentionally keeping Palomar lightweight in order to allow for reliable maintainance and quality control.

Lars Warren Ericson · 30 August 2026

In formalizing whole papers like Scott **1964** *Measurement Structures and Linear Inequalities* with GPT 5.6 Sol, my process is to do a reliable OCR process followed by a single prompt of form "Formalize all definitions, notations, and theorems in Lean 4 for text paper.md". Then run a Palomar registration, which is a way to put the formalized paper out into the world. Palomar doesn't index the formalization in Google Scholar. The process itself doesn't merit a paper in arXiv because the act

of pushing the button on an autoformalization is no longer notable. I can put the formalization and a short methodology note in HAL.science, which does index in Google Scholar and allows a wider range of artifacts than arXiv. What do you recommend as a template for methodology notes, and as the best way to get the pairing of the autoformalization with the original paper into Google Scholar?

Anonymous · 2 September 2026

I am hoping soon Palomar becomes a mainstream academic research registry. It already supports bib citation, that means the board has a vision to make it mainstream registry may be not soon.

Lars Warren Ericson · 4 September 2026

Cooldown penalties are excessive. I am up to 5 days between submits for a repo that was blocked by system problems that needed to be fixed by Palomar developer pull requests. The net effect is punitive gatekeeping. Is this the desired effect?

arnaudmayeuxmath · 5 September 2026

I have built an index that runs from published mathematics to Lean: the entry point is a paper, not a repository.
It covers roughly 5M records, with formalization artifacts attached to the papers they formalize (e.g. Palomar entries, arXiv formalization papers). Each link is verified individually against the authors of the source paper. Every record carries a permanent code, such as MiNDS3MDGRHZCD5.
https://mathinsid.org
It is still early. Such an index should become more useful as the formalized corpus grows in size and level.
Corrections and suggestions are very welcome.
Method: arXiv:2606.11430 (v1 9 Jun 2026)

Lars Warren Ericson · 6 September 2026

Arnaud, please index HAL.science preprints as well as arXiv preprints. arXiv is inconsistent about accepting formalization notes. Of two notes both describing formalizations of two papers by the same author on the same subject, one could be deemed to be in scope for the arXiv category and the other out of scope, when both formalization notes are similarly prepared and both Lean codes accepted by Palomar. I am putting new formalization notes in HAL.science from now on. Your site picked up two of my arXiv Lean papers. I'm worried it won't pick up any archived in HAL.science.

arnaudmayeuxmath · 9 September 2026

Lars, thank you for the note. MathINsiD is designed to index HAL papers, subject to the same conditions as any other source: a real published or preprint paper, with formalization artifacts individually verified. More generally: MathINsiD aims to stay open to any proof system and any index, current or future, including but not limited to Lean, Isabelle, Coq, arXiv, HAL, Palomar, and mathlib, all of which we value. The totem here is the published literature; MathINsiD will adapt on the formalization side as that landscape keeps advancing. The entry point is the paper; verified formalization artifacts pointing to it can come from any serious source. Please click "Contact" on the site to submit an entry if needed.